

KDA as a Gated Delta Net

Finding a NaN in Tokamax's Kimi Delta Attention kernel, fixing it, and measuring the result

Majid Taheri · 3 September 2026 · branch Majid-Taheri/tokamax @ gdn-scalar-gate

Summary. Routing Qwen3.5's Gated Delta Net through Tokamax's KDA Pallas kernel produces `loss: nan` at step 0. The cause is a numerical defect in the kernel, not in the integration: to batch a matmul, the kernel splits an exponent that is always ≤ 0 into two factors, one of which overflows to infinity while the other underflows to zero. The defect is present in `openxla/tokamax` main today. A fix was implemented behind a new `per_channel_gate` flag, validated forward and backward against the reference implementation, and benchmarked. The fix is correct; it is not faster.

Two performance predictions made during this work were wrong and are retracted in section 8. The measurements, not the reasoning, are what should be relied on.

1. What went wrong

Qwen3.5's Gated Delta Net (GDN) and Kimi Delta Attention (KDA) implement the same chunked delta rule. They differ in one respect: KDA's forget gate carries an independent value for each key channel, while GDN's is a single scalar per head and token. An adapter was written that broadcasts GDN's scalar gate across all 128 channels and calls KDA.

At reduced scale this trains correctly. At the full training configuration it does not:

Run	Step 0 loss	Outcome
Pure-JAX control (unchanged path)	12.923	reaches step 9 at 11.901
KDA kernel, identical config	nan	aborted at step 0

A loss of 12.923 is approximately $\ln(\text{vocabulary size})$, which is the expected value at random initialisation. The control establishes that the model, data, sharding and optimiser are all sound, so the fault lies in the kernel path alone.

2. Root cause

In `pallas_mosaic_tpu_fwd_kernel.py`, the intra-chunk attention term the kernel needs is

```
Aqk[r,t] = sum_k q[r,k] * k[t,k] * exp2(g[r,k] - g_cumsum[t,k])    for t <= r
```

The gate accumulates negatively, so for every causal pair the exponent $g[r] - g_cumsum[t]$ is at most zero and the whole quantity is bounded by one. It is perfectly safe as written.

But the gate depends on the key channel, so it cannot be lifted out of the sum over k . To fold it into the operands and still perform a single matmul, the kernel factors the exponent around a reference row:

```
ref_idx = BC // 2 if safe_gate else 0
gn_ref  = g_i[:, ref_idx : ref_idx + 1, :]

q_eg      = q_i * exp2(g_i - gn_ref)          # first factor
diff_j_safe = (gn_ref - g_cumsum) * valid_j    # POSITIVE for causal t
k_eng_full = k_f32 * exp2(diff_j_safe) * valid_j # second factor
```

The product of the two factors is bounded. The factors individually are not. The second grows with the cumulative gate drop across the block and overflows to infinity; the first underflows to zero. **$\text{inf} \times 0 = \text{NaN}$** .

The code comment at that site guards the anti-causal rows for exactly this reason. The causal direction has the same problem and is not guarded. The `safe_gate` option moves the reference row from index 0 to `BC//2`, which halves the exponent range rather than bounding it, and it was already enabled throughout.

The backward carries the same defect in its diagonal-block term (`pallas_mosaic_tpu_bwd_kernel.py`), which re-centres on the block maximum so that `col_d = exp2(g_max - g_b)` is positive. Its off-diagonal paths are already safe, because both of their exponents are ≤ 0 by construction.

3. Confirming the mechanism

Test A — does the threshold match the arithmetic? $BC = 16$ and the reference sits at $BC//2 = 8$, so the split spans eight tokens of gate decay against a float32 $\exp_2$ ceiling of 2^{128} . That predicts failure once the gate falls by more than $128/8 = 16$ in log2 units per token. Sweeping the gate on TPU7x:

Gate scale	min g / token	chunk drop (log2)	avg log2 / token	Mosaic result
0.25 – 21	-0.25 to -21	7.9 – 667	up to 10.4	finite, matches reference
30	-30	953	14.9	NaN 5.8%
40	-40	1271	19.9	NaN 28.5%
60	-60	1907	29.8	NaN 46.4%
100	-100	3178	49.7	NaN 73.2%

The edge falls between 10.4 and 14.9 log2 per token, against a predicted 16. The failure is also per-head, not random: `A_log` is a per-head parameter, so the fraction of NaN outputs tracks the fraction of heads whose gate exceeds the threshold.

Test B — is it hardware- or precision-specific? No. A standalone reproduction on one chip ($H=4, B=1, T=512, D=128$) gives identical results in float32 and bfloat16:

A_max	min g / token	chunk drop (log2)	Mosaic	XLA reference
0.5	-1.59	55.8	finite	finite
2.0	-6.37	223.1	finite	finite
8.0	-25.50	892.5	finite	finite
16.0	-50.99	1784.9	NaN 50.0%	finite

Qwen3.5 initialises `A_log = log(Uniform(0, 16))` with `dt_bias = 1`, which is the last row. The model lands in the failing range by default. The backward behaves the same way: at `A_max = 16` all five gradients are NaN in 50% of entries while the XLA reference stays finite.

Test C — does the bug also affect KDA used as KDA? Yes, but only in one of its two gate modes. KDA's own activation (`reference.py`) is:

```
lower_bound is None -> g = -A * softplus(g_f)      # unbounded below
lower_bound = L      -> g = L * sigmoid(A * g_f)    # saturates at L
```

Mode	A_max	Mosaic result
unbounded (<code>lower_bound = None</code>)	0.5	finite
unbounded	8.0	NaN 50%
unbounded	16.0	NaN 50%
bounded (<code>lower_bound = -1.0</code>)	0.5 / 16 / 64	finite at every scale

The bounded gate is immune by construction: it saturates at `lower_bound` per token however large `A` becomes, so the exponent can never reach the overflow range. This very likely explains why the authors never encountered the defect. It is also why `lower_bound` is not a usable workaround for GDN: $L \cdot \text{sigmoid}(A \cdot g)$ is a different function from $-A \cdot \text{softplus}(g)$, not a clamp of it, so adopting it would change the model.

4. Isolating the integration from the kernel

Before blaming the kernel, the MaxText integration was bisected to rule out a wiring error. With the gate initialisation tamed so the forward stays inside the kernel's range, the KDA path trains correctly at every reduced configuration:

Layers	Batch / device	Sequence	Result over 10 steps
4	1	1024	12.913 → 12.499
4	1	4096	12.925 → 12.679
4	4	4096	12.923 → 12.776
16	4	4096	12.922 → 12.508
48	1	4096	12.923 → 11.192
48	4	4096	step 0 correct, step 1 nan

Step 0 matching the pure-JAX control to three decimals establishes that the adapter's layout, GQA expansion, gate convention, state handling, bfloat16 casting and shard_map wrapping are all correct. The remaining failure in the last row is discussed in section 9.

5. The fix

A scalar gate does not need the factorisation at all. When the decay does not depend on the key channel it comes out of the sum entirely, so the matmul can run ungated and the decay applied afterwards as a chunk-by-chunk matrix:

```
gs    = g_cumsum[:, :, 0]                # [MB, BT]
diff  = gs[:, :, None] - gs[:, None, :]  # [MB, BT, BT]
decay = exp2(where(causal, diff, 0.0))    # exponent always <= 0
qk    = dot(q, k, contracting over k)    # gate-free matmul
Aqk   = where(causal, qk * decay * scale, 0.0)
```

The exponent is now formed as a single difference, masked to the causal half where it is guaranteed non-positive, so `exp2` lands in $(0, 1]$ and cannot overflow. Under the mask the two formulations are algebraically identical — this is a regrouping, not an approximation. The same regrouping replaces the `g_max` re-centring in the backward's diagonal block.

The behaviour is selected by a new op parameter, `per_channel_gate`, threaded through `api` → `base` → `op` → `forward` and `backward` kernels. When true, KDA behaves exactly as before. When false, the gate may additionally be supplied at width 1 instead of width 128, and is kept narrow through the input tensor, the `g_cumsum` residual, stage 3 and the backward.

Changes on the branch

Commit	What it does
Add <code>per_channel_gate</code>	Scalar-gate forward path
Extend to the backward	Same regrouping in the diagonal-block term
Declare static	<code>per_channel_gate</code> added to <code>chunk_kda_bwd_custom</code> 's <code>static_argnames</code>
Accept a width-1 gate	Removes the 128× broadcast end to end
Validation scripts	<code>gdn_validation/</code> — six harnesses plus a README
Revert VMEM widening	Undoes a change justified by a hypothesis later disproved

6. Correctness results

All results below are TPU v5, float32, H=4 B=1 T=512 D=128, comparing the Mosaic kernel against Tokamax's own XLA reference on byte-identical inputs. Figures are maximum relative error. A_max = 0.5 is a mild gate; A_max = 16 is Qwen3.5's actual initialisation.

Forward

A_max	per_channel_gate	Mosaic	vs XLA reference
0.5	True	finite	2.62e-03
0.5	False	finite	1.73e-03
16.0	True	NaN 50.0%	—
16.0	False	finite	1.85e-03

Backward — all five gradients

A_max	mode	out	d_query	d_key	d_value	d_gate	d_beta
0.5	True	2.6e-03	4.9e-04	6.6e-04	1.7e-03	8.8e-04	1.7e-03
0.5	False	1.7e-03	5.1e-04	5.5e-04	1.7e-03	8.4e-04	2.4e-03
16.0	True	NaN	NaN	NaN	NaN	NaN	NaN
16.0	False	1.8e-03	6.0e-04	1.1e-03	1.9e-03	1.6e-03	2.0e-03

Width-1 gate

Three configurations, all against the reference. Rows two and three must agree: the only difference between them is whether the same number is stored once or 128 times.

A_max	per_channel	gate width	gate size	out	worst gradient
16.0	True	128	1.0 MB	NaN 50%	NaN 50%
16.0	False	128	1.0 MB	1.8e-03	2.0e-03
16.0	False	1	0.008 MB	1.8e-03	2.0e-03

The two scalar rows agree to the printed digit on every quantity except d_gate (1.6e-03 against 2.0e-03). That is expected rather than a discrepancy: with a narrow gate d_gate is a sum over 128 channels rather than 128 separate values, so it rounds differently. Both are well within tolerance of the reference.

Regression check

In every table above, the per_channel_gate=True rows are unchanged from stock upstream, including still failing at A_max = 16. That is the intended behaviour: the KDA path was not to be altered, and it was not.

7. Performance results

TPU v5, H=32 B=4 T=4096 D=128, bfloat16, mild gate, median of ten iterations after warm-up. The gate is deliberately mild so both paths compute the same thing — timing a kernel that is producing NaN would be meaningless.

Configuration	gate tensor	forward	forward + backward
per-channel (stock behaviour)	256.0 MB	2.91 ms	9.37 ms
scalar regrouping, gate still broadcast	256.0 MB	2.83 ms	9.32 ms
scalar regrouping, width-1 gate	2.0 MB	3.25 ms	9.81 ms

Comparison	forward	forward + backward
regrouping alone, against stock	1.03×	1.00×
width-1 gate, against stock	0.90×	0.96×

The memory objective is met exactly: 128× smaller, and the `g_cumsum` residual shrinks by the same factor (visible in the autotuning key as `g_cumsum: f32[32, 4, 4096, 1]`). The time cost is real: roughly 10% in the forward and 4% end to end.

8. Two predictions that were wrong

Both are recorded because the reasoning behind them was plausible and would otherwise be repeated.

Prediction 1 — the regrouping would be faster. The scalar path performs the same matmul FLOPs, roughly ten times fewer `exp2` operations, and materialises no per-sub-block temporaries. Measured: 1.03× forward, 1.00× overall. The error was reasoning about operation counts without asking which resource was actually saturated. This kernel is dominated by matmuls and memory traffic, and the change altered neither; the transcendentals were never the bottleneck.

Prediction 2 — the width-1 slowdown was a lane-broadcast cost. On TPU the minor axis is the 128-lane dimension, so a width-1 tensor wastes the vector registers and every elementwise use appears to need a broadcast. The backward, which widens the gate in VMEM immediately after loading it, showed no regression, while the forward, which left it narrow, was 10% slower — an apparently clean natural experiment. The forward was changed to widen in VMEM as well. Measured: 3.27 ms became 3.25 ms, which is noise. The hypothesis was wrong, the change was reverted, and the cost is more likely in the width-1 block itself — Mosaic probably pads it to a full tile regardless, so the HBM saving buys nothing on a kernel that is not HBM-bound.

9. What remains open

The full-configuration NaN is not explained. At 48 layers, batch 4 per device and sequence 4096, MaxText produces a correct step 0 and then nan at step 1. Ruled out by experiment, not by argument: gate magnitude (taming to `A_max = 0.05` does not help), `dtype`, `shard_map`, `remat_policy=full`, the scoped VMEM flag, sequence length, depth alone and batch alone. In isolation at exactly those per-shard shapes all six gradients are finite and within `bfloat16` noise of the reference. Only the three factors together fail.

The MaxText adapter has not been pointed at the fix. It still broadcasts the gate to width 128 and leaves `per_channel_gate` at its default. Wiring it to `False` and rerunning is the test that decides whether this work unblocks training.

Nothing has been benchmarked on TPU7x. Every timing here is from v5, a generation upstream explicitly declares unsupported (`supported_on()` requires generation ≥ 6 ; the validation scripts honour `KDA_BYPASS_DEVICE_CHECK=1`). The correctness findings are hardware-independent — the defect is a `float32` exponent range problem, and it also reproduces on TPU7x through MaxText — but the timings are not transferable.

The defect should be reported upstream. It is present in `openxla/tokamax` main, not only in the unmerged pull request, so anyone using KDA with an unbounded gate is exposed.

10. Reproducing this

Everything, including the validation harnesses, is on the branch.

```
git clone -b gdn-scalar-gate \
    https://github.com/Majid-TaHERi/tokamax.git tokamax-src
cd tokamax-src
pip install -e . --no-deps
pip install xprof --no-deps          # new dependency on main

export KDA_BYPASS_DEVICE_CHECK=1    # only needed below TPU generation 6

python3 gdn_validation/test_per_channel_gate.py      # forward
python3 gdn_validation/test_per_channel_gate_grad.py # gradients
python3 gdn_validation/test_narrow_gate.py          # width-1 gate
python3 gdn_validation/bench_per_channel_gate.py     # timings

python3 gdn_validation/kda_gate_nan_repro.py        # bug on stock upstream
python3 gdn_validation/kda_native_gate_test.py      # KDA's own gate modes
```

Each test exits 0 on success and 1 on failure. `git checkout main` inside the checkout gives the stock kernel for comparison.

Practical notes

Trap	Consequence
JAX_DEFAULT_MATMUL_PRECISION =highest	Makes the KDA backward fail to compile in bfloat16 ("Bad rhs type: 256, 256"). A harness artefact, not a kernel bug.
jaxtyping binds K from the query	Every annotation on the gate's path — parameters and return tuples, in both kernels — must declare its own width or a width-1 gate is rejected while the arithmetic is fine.
Upstream restricts KDA to generation ≥ 6	TPU v5 needs the documented device-check bypass.
pip installs openxla main, not the antgroup branch	The two differ; patches written against the wrong base will not apply.

11. Conclusion

The KDA Mosaic kernel returns NaN for gates steeper than roughly 11 nats per token, in both the forward and the backward, because it splits a bounded exponent into two unbounded factors. Qwen3.5's Gated Delta Net needs about 21 nats per token at initialisation, so it fails immediately. The defect is in upstream main and also affects KDA's own unbounded-gate mode; only its bounded-gate mode is immune.

Specialising the kernel for a scalar gate removes the defect by construction rather than patching around it, agrees with the reference to within 2e-03 on every output and gradient at Qwen3.5's actual gate, leaves the per-channel path untouched, and optionally reduces the gate's memory by 128×. It is not faster — the regrouping is neutral and the width-1 gate costs about 4% end to end on v5.

The value of this work is that it makes the configuration run at all. Whether it makes training work is a separate question, and answering it requires pointing the MaxText adapter at the new flag and repeating the 48-layer run.

Branch: github.com/Majid-TaHERi/tokamax, branch `gdn-scalar-gate`, rebased on `openxla/tokamax` main. Measurements dated 3 September 2026.